# Clean Up the Parking Lot

Code Smells, Refactoring & Program Comprehension — identify code smells and refactor legacy Java code without changing its behaviour.

| Submission date | Work mode | Course |
|---|---|---|
| Monday, 26 October 2026 | Pairs (2 members) | University of Central Punjab · Faculty of Information Technology |



| Name | Roll No | Name | Roll No |
|---|---|---|---|
| Muhammad Arham | L1F23BSSE0372 | | |



## The scenario

The university parking office prints parking tickets with the **Version 1.0** program below. It compiles and gives the right output, but nobody wants to touch it. The job is to re-engineer it into a clean **Version 2.0** that prints *exactly the same output*.

```
class vehicle {
    public String n;      // plate number
    public int t;         // type: 1 = car, 2 = bike
    public int h;         // hours parked
    public String z;      // not used anymore

    // this method prints the parking ticket
    void process() {
        int c = 0;
        if (t == 1) {
            c = h * 50;
            if (c > 1000) c = 1000;
        } else if (t == 2) {
            c = h * 20;
            if (c > 1000) c = 1000;
        }
        System.out.println("------------------");
        System.out.println("Plate : " + n);
        System.out.println("Hours : " + h);
        System.out.println("Charge : Rs. " + c);
    }

    void printPass() {
        System.out.println("------------------");
        System.out.println("Plate : " + n);
        System.out.println("Hours : " + h);
        if (t == 1) System.out.println("Type : Car");
        else if (t == 2) System.out.println("Type : Bike");
    }

    void oldDisplay() {   // nobody calls this
        System.out.println(n);
    }
}

public class ParkingSystem {
    public static void main(String[] args) {
        vehicle v1 = new vehicle();
        v1.n = "LEA-1234"; v1.t = 1; v1.h = 6;
        v1.process();
        v1.printPass();

        vehicle v2 = new vehicle();
        v2.n = "LHR-5678"; v2.t = 2; v2.h = 60;
        v2.process();
        v2.printPass();
    }
}
```

**Expected output (must stay identical after refactoring)**

```
------------------
Plate : LEA-1234
Hours : 6
Charge : Rs. 300
------------------
Plate : LEA-1234
Hours : 6
Type : Car
------------------
Plate : LHR-5678
Hours : 60
Charge : Rs. 1000
------------------
Plate : LHR-5678
Hours : 60
Type : Bike
```

## 0 Warm-up: what is a code smell?

A code smell is a surface sign in the code that usually points to a deeper design problem. The program still works, so a smell is not a bug — but it makes the code harder to read, change and extend.

SMELLY

```
if (age > 18) {{ ... }}
double t = p * 0.17;
```

CLEAN

```
if (age > ADULT_AGE) {{ ... }}
double tax = price * TAX_RATE;
```

**0.1 Both versions give the same result. In one line, why is the right-hand version better?**

> **Ans:** `ADULT_AGE` and `TAX_RATE` say what the numbers mean, so the reader doesn't have to guess the intent of `18` and `0.17`, and each value lives in one named place instead of being scattered as an unexplained literal.

**0.2 Is a code smell a bug? Yes / <u>No</u>**

> **Ans:** No. The program still compiles and produces the right output. A smell is a surface sign of a deeper design problem that makes the code harder to read, change and extend.

## A Spot the smells in Version 1.0

| # | Smell name | Line(s) | Why does it hurt? |
|---|---|---|---|
| 1 | Naming-convention violation | 1 | Java class names are PascalCase; `vehicle` looks like a variable. |
| 2 | Poor / cryptic naming | 2–5 (also `c` on 9) | `n, t, h, z, c` mean nothing on their own — you need the comments to read the code. |
| 3 | Public fields (no encapsulation) | 2–5 | Anyone can write `v1.h = -5;` ; the object cannot protect its own state. |
| 4 | Unused field | 5 | `z` is never read or written — it only adds noise. |
| 5 | Magic numbers | 11, 12, 14, 15 | 50, 20 and 1000 have no explanation; a rate change means hunting every occurrence. |
| 6 | Duplicate code | 17–19 vs 24–26 | The same three print lines appear twice, so a formatting fix must be made twice. |
| 7 | Long method / method doing two jobs | 8–21 | `process` both calculates the charge and prints the ticket. |
| 8 | Type code with repeated conditionals (switch-statement smell) | 10–16 and 27–28 | `t == 1 … else if (t == 2)` is repeated in two methods, so a new type must be added twice. |
| 9 | Dead code | 31–33 | `oldDisplay` is never called. |
| 10 | Meaningless method name | 8, 23 | `process` and `printPass` do not say what they do ( `process` actually prints a ticket). |
| 11 | Comments used as deodorant | 2–5, 7, 31 | Comments explain cryptic names ( `n` , `t` , `h` ) instead of the names being clear. |
| 12 | No constructor (object built field by field) | 38–39, 43–44 | Each object is assembled in several lines, so a field can easily be forgotten. |

## B Refactor step by step

> **Golden rule:** refactoring changes the structure of code, never its behaviour. One small change at a time; after each step the output must still match the expected output.

### B1 · Rename (class, field, method, variable)

| Old | New |
|---|---|
| `vehicle` (class) | `Vehicle` |
| `n` (field) | `plateNumber` |
| `t` (field) | `vehicleType` (removed entirely in B5) |
| `h` (field) | `hoursParked` |
| `c` (local) | `charge` |
| `process` | `printTicket` |
| `printPass` | `printPassDetails` |
| `oldDisplay` | (deleted in B6) |

### B2 · Encapsulate Field + Introduce Constructor

**Ans:** Make `plateNumber` and `hoursParked` `private final` and add a constructor `Vehicle(String plateNumber, int hoursParked)`. The three-line set-up of each object in `main` then collapses into one line, e.g. `Vehicle v1 = new Car("LEA-1234", 6);`

### B3 · Replace Magic Number with Named Constant

**Ans:** constants introduced:

```
static final int MAX_CHARGE = 1000;     // in Vehicle, shared by all types
static final int RATE_PER_HOUR = 50;    // in Car
static final int RATE_PER_HOUR = 20;    // in Bike
```

`MAX_CHARGE` is shared by all vehicles; the rates differ per type, which is why they live in the subclasses after B5.

### B4 · Extract Method

**Ans:**

- `printHeader()` — pulls the three repeated print lines (17–19 and 24–26) into one private method called by both `printTicket` and `printPassDetails`.
- `calculateCharge()` — separates calculating from printing.
- The cap becomes one line: `Math.min(hoursParked * getRatePerHour(), MAX_CHARGE)`.

### B5 · Replace Type Code with Subclasses

**Ans:** `Vehicle` becomes `abstract` and declares `int getRatePerHour()` and `String getTypeName()`. `Car` answers 50 / "Car", `Bike` answers 20 / "Bike". Every `if (t == …)` disappears.

### B6 · Remove Dead Code

**Ans:** `oldDisplay` (lines 31–33) and the field `z` (line 5) are deleted — Section E2 proves nothing references them.

## Version 2.0 — final code

```java
// File: ParkingSystem.java
abstract class Vehicle {
    private final String plateNumber;
    private final int hoursParked;

    static final int MAX_CHARGE = 1000;

    Vehicle(String plateNumber, int hoursParked) {
        this.plateNumber = plateNumber;
        this.hoursParked = hoursParked;
    }

    abstract int getRatePerHour();
    abstract String getTypeName();

    int calculateCharge() {
        return Math.min(hoursParked * getRatePerHour(), MAX_CHARGE);
    }

    private void printHeader() {
        System.out.println("------------------");
        System.out.println("Plate : " + plateNumber);
        System.out.println("Hours : " + hoursParked);
    }

    void printTicket() {
        printHeader();
        System.out.println("Charge : Rs. " + calculateCharge());
    }

    void printPassDetails() {
        printHeader();
        System.out.println("Type : " + getTypeName());
    }
}

class Car extends Vehicle {
    static final int RATE_PER_HOUR = 50;

    Car(String plateNumber, int hoursParked) {
        super(plateNumber, hoursParked);
    }

    @Override int getRatePerHour() { return RATE_PER_HOUR; }
    @Override String getTypeName() { return "Car"; }
}

class Bike extends Vehicle {
    static final int RATE_PER_HOUR = 20;

    Bike(String plateNumber, int hoursParked) {
        super(plateNumber, hoursParked);
    }

    @Override int getRatePerHour() { return RATE_PER_HOUR; }
    @Override String getTypeName() { return "Bike"; }
}

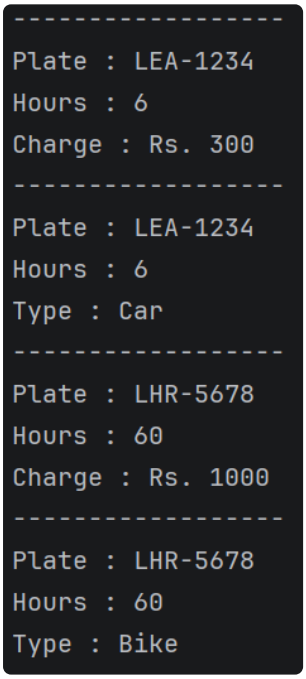
public class ParkingSystem {
    public static void main(String[] args) {
        Vehicle v1 = new Car("LEA-1234", 6);
        v1.printTicket();
        v1.printPassDetails();

        Vehicle v2 = new Bike("LHR-5678", 60);
        v2.printTicket();
        v2.printPassDetails();
    }
}
```

**Console output (matches the expected output line for line: ☑ Yes ☐ Not yet)**

```
------------------
Plate : LEA-1234
Hours : 6
Charge : Rs. 300
------------------
Plate : LEA-1234
Hours : 6
Type : Car
------------------
Plate : LHR-5678
Hours : 60
Charge : Rs. 1000
------------------
Plate : LHR-5678
Hours : 60
Type : Bike
```

Screenshot of the console output (IDE run):




## C Smell-detection tools

| Tool | Letter |
|---|---|
| 1. PMD | **b** |
| 2. Checkstyle | **d** |
| 3. SonarQube | **c** |
| 4. JDeodorant | **a** |

**C.1 Which tool would flag line 1 ( `class vehicle` ) and line 12 ( `1000` )?**

> **Ans:** Checkstyle — it enforces naming conventions and magic-number rules.

**C.2 Which smell from Section A would JDeodorant be best at finding?**

> **Ans:** Type code with repeated conditionals (the `t == 1` / `t == 2` chains on lines 10–16 and 27–28) — the type-checking switch smell JDeodorant specialises in.

**C.3 PMD's "unused private method" rule would not flag `oldDisplay()`. Why not?**

> **Ans:** `oldDisplay` on line 31 has no access modifier, so it is package-private, not private. The rule only fires for private methods, since only then can the tool be sure nothing outside the class could call it.

## D Benefits check: apply a change request

| Change request | Edits in Version 1.0 | Edits in Version 2.0 |
|---|---|---|
| CR-1: Car rate becomes Rs. 60 per hour | 1 (line 11) | 1 (`RATE_PER_HOUR` in `Car`) |
| CR-2: Maximum charge becomes Rs. 1500 | 2 (lines 12 and 15) | 1 (`MAX_CHARGE` in `Vehicle`) |
| CR-3: Add new vehicle type, Truck, at Rs. 100 per hour | 3 (an `else if` in `process` lines 10–16, another in `printPass` lines 27–28, plus the set-up in `main`) | 0 edits to existing code — just 1 new subclass, `Truck` |

**D.1 Three benefits of the refactoring:**

> 1. **Readability** — the code says `calculateCharge` and `getRatePerHour` instead of `c = h * 50`, so it reads without the old comments.
> 2. **Maintainability** — each business rule lives in exactly one place (rate in the subclass, cap in `MAX_CHARGE`), so there are fewer places for a bug to hide.
> 3. **Extensibility (Open/Closed)** — a new type such as Truck is one new class plugged into the abstract parent, with no edits to tested code and no chance of forgetting a second `else if`.

## E Program comprehension: chunking and cross-referencing

**Chunking:** grouping lines you recognise as one idea and naming that group, so you read 5 ideas instead of 20 lines. **Cross-referencing:** linking a name's definition to every place it is used (IDE: Ctrl+Click, Find Usages, Call Hierarchy; or grep).

### E1 · Chunking

```
1   void exitParking(String plate, String slotId) {
2       if (plate == null || slotId == null) {
3           System.out.println("Invalid input");
4           return;
5       }
6       Slot slot = lot.find(slotId);
7       Vehicle vehicle = registry.find(plate);
8       slot.setFree(true);
9       slot.setVehicle(null);
10      int charge = vehicle.calculateCharge();
11      if (charge > 0) {
12          wallet.deduct(plate, charge);
13          sms.send(vehicle.getPhone(), "Paid Rs. " + charge);
14      }
15      log.write(plate + " left slot " + slotId);
16  }
```

| Lines | Chunk label |
|---|---|
| 2–5 | Validate input |
| 6–7 | Find slot, vehicle |
| 8–9 | Release the slot |
| 10–14 | Charge and notify |
| 15 | Write log entry |

### E2 · Cross-referencing

| Name | Defined at line | Used at line(s) |
|---|---|---|
| `t` | 3 | 10, 13, 27, 28 (set at 39, 44) |
| `h` | 4 | 11, 14, 19, 26 (set at 39, 44) |
| `process()` | 8 | 40, 45 |
| `oldDisplay()` | 31 | none — no call sites |
| the number `1000` | not defined (literal) | 12, 15 |

**E2.1 What does your table prove about `oldDisplay()`?**

> **Ans:** It has no call sites anywhere in the program, so it is dead code and safe to delete without changing behaviour.

**E2.2 Why was searching for `h` painful in a text search?**

> **Ans:** A single letter matches inside unrelated words and comments (*this*, *Charge*, *hours*, *the*), so results are flooded with noise and you cannot tell the field's real uses from accidental matches.

### E3 · Put them together

> **Ans:** In a 200-file system I would start from the entry point and cross-reference (Ctrl+Click, Find Usages, Call Hierarchy) to follow the call chain down to the method that actually computes the parking charge, then chunk each long method on the way into labelled groups so I read a handful of named ideas instead of hundreds of lines. Cross-referencing tells me *where* the charge logic lives; chunking tells me *what* each part of it does.

## ✓ Reflection

- **One smell I will now always notice:** repeated `if (type == …)` chains — the type-code switch smell — because a new type must be added in several places.
- **One refactoring I can now apply:** Extract Method — pulling duplicated print lines and a calculation out into small, named, single-purpose methods.